SNAPCLAIM · ONBOARDING FAQ · META GLOBAL AI DEVELOPER HACKATHON

Your Questions, Answered

Hi Rizgar — thanks for the thoughtful questions. Here's where SnapClaim stands, what I cover, and where you'd fit in.

1 Who's on the team right now?

PERSON	ROLE	COVERS
Parithosh Varma (me)	Solo founder → team lead	Product spec, implementation plan, measurement engine (deterministic math + receipts + evidence gate), Meta model integration, Cloudflare Worker, browser/PWA engineering
Rizgar Ozan (invited)	Write collaborator · pending acceptance	Your pick: CV/vision (SAM 3.1 prompting, mask quality, scale-reference calibration) **or** frontend/renderer/video (PWA capture, SVG/PDF renderer, 3-minute demo edit)

- Repo is public: **github.com/Parithosh-Varma/snapclaim**.
- Invite sent to **RizgarOzan** — accept at github.com/Parithosh-Varma/snapclaim/invitations

2 Is there a repo yet, and what works?

- **Yes — public repo.** Day 1 core shipped and committed: deterministic measurement engine (masks → area % → calibrated crack length → severity bands), receipt ledger with deterministic IDs, and an evidence gate that throws instead of shipping a number a receipt can't back.
- **24/24 tests pass, `tsc --noEmit` clean, $0 API spend.** Mock-mode pipeline runs entirely offline against fixture data (synthetic wall-crack stats + coin reference).
- **Not yet live:** real model calls (SAM 3.1 / Muse Spark / Spark Open / voice transcribe), PWA capture UI, SVG/PDF renderer, and real damage photos.
- **Tech stack:** TypeScript everywhere · Cloudflare Worker proxy (keys server-side) · static deploy on Cloudflare Pages · Apache-2.0.

3 What does the final demo look like, and which track?

Demo arc (3 minutes): real crack on a real wall → phone photo → SAM 3.1 masks bloom → one Hinglish voice line → annexure writes itself with green receipts → one finding flips **DISPUTED** (skeptic can't reproduce it, scale marker occluded) → human corrects → green, co-signed → receipts page + replay. Close: "Computed, not generated — because a claim is an argument, and arguments need evidence."

Track: Meta Global AI Developer Hackathon, **Indie track**, Oct 26 – Nov 4, 2026. Deadline Nov 4, 9:00 PM ET = Nov 5, 7:30 AM IST; submit Nov 4 evening IST. Feature freeze Nov 2.

TypeScript · Browser PWA · SAM 3.1 · Muse Spark · Open weights · 8-day sprint

4 Hours per day until Nov 4?

~2–3 h/day, evenings IST, Oct 27 → Nov 4. Async-first, one 15-minute sync per day. The core is already done, so your first week is lighter: trial call + one small PR, then your owned area ramps up as Days 2–4 go live.

5 Would I own the eval end-to-end?

> **Yes — if you want it.** The eval is yours to design and run:
> • **Gap benchmark:** model-only qualitative estimate vs measured numbers (the guessed-vs-measured gap is our headline metric).
> • **Held-out split:** fixture set published before tuning; weak rows printed, not buried.
> • **Scoring & metrics page:** percent vs percent-only mode, skeptic agreement rate, replayability check.
> • Mock-mode CI runs zero spend; one live run archived as a JSON artifact.
> If you'd rather support someone else on this, that's fine too — I'll adapt. Either way, eval is not an afterthought; it's the credibility of the demo.

How we'd work together

- **Trial first:** 20-minute call + one small PR/issue before we commit to the team.
- **Prize split:** agreed in writing before submission.
- **Credits:** claimed individually via official forms — no pooling, no account sharing.
- **Code is public from Day 1;** build logs honest and dated in `DEV-LOG.md`.

Next: tell me **which area you'd own** (CV/vision, frontend/renderer/video, or eval) and **one thing you've shipped**. First reply gets the plan PDF and a call slot.